zarrcurator Documentation

version

Kameswarrao Modali

September 30, 2026

Contents

zarrcurator Documentation **1**

- Overview 1
  - Build Documentation 1
  - Regenerate API Docstring Page 1
  - Object Store Credentials 1
  - Synthetic Test Stores 1
- Store Layouts 2
  - Nested groups 2
  - Sharding 2
  - Kerchunk references 3
  - Synthetic test stores 3
- Known Issues and Limitations 3
  - Known issues 3
  - Limitations 4
  - Worked around 4
- zarrcurator API Documentation 4
  - Module: **init** 4
    - Functions 4
      - enable_logging() 4
  - Module: benchmark 5
    - Functions 5
      - create_synthetic_store() 5
      - benchmark_diagnose() 5
      - benchmark_dispatch() 5
      - parse_args() 5
      - main() 5
    - Classes 5
      - _BenchDistributor 5
  - Module: chunkkeys 5
    - Functions 5
      - parseChunkKey() 5
  - Module: cli 5
    - Functions 6
      - configure_logging() 6
      - main() 6
  - Module: config 6
    - Classes 6
      - configVars 6
        - Methods of configVars 6
  - Module: core 7
    - Functions 8
      - hello() 8
      - helloFrom() 8
  - Module: diagnoser 8

Classes 8
Diagnoser 8
Methods of Diagnoser 8
Module: dispatcher 9
Classes 9
Dispatcher 9
Methods of Dispatcher 9
Module: displayer 9
Classes 10
Displayer 10
Methods of Displayer 10
Module: distributor 10
Classes 10
Distributor 10
Methods of Distributor 10
Module: oceanVarsYAML 11
Module: storetree 11
Functions 11
isZarrGroupDir() 11
listArrayPaths() 12
groupAncestors() 12
Module: subgroup 12
Functions 12
subGroup_to_dict() 12
subGroup_from_dict() 12
Classes 12
SubGroup 12
Methods of SubGroup 12
Module: synthetic_store 13
Functions 13
store_variant_tag() 13
create_synthetic_experiment() 13
run_smoke_pipeline() 13
parse_args() 13
main() 13
Module: utils 13
Functions 13
usage() 13
saveConfigToFile() 14
loadConfigFromFile() 14
setObjectStoreCredentials() 14
updateStoretoConfig() 14
findMissingChunksInStore() 14
initialize() 14
diagnose() 14
diagnoseStore() 15

display() 15
running_in_jupyter() 15
displayStoreSunburst() 15
displayZarrStoreVariables() 15
distribute() 15
distributeStoreToSubStores() 15
checkStoreOpensWithXarray() 15
dispatchStoreToTarStores() 16
dispatch() 16
createKerchunkReferenceFileForTar() 16
mergeKerchunkReferences() 16
convertKerchunkReferenceFilesOfZarrStoreToParquet() 16
updateArchBasePath() 16
updateTarToTapeMapper() 17
mapTapeIDsToFrequencySubGroupDict() 17
displayZarrStoresTarTapes() 17
displayZarrStoreVarTarTapes() 17
Module: zarrmetadata 17
Functions 17
v3CodecsToV2() 17
v3MetadataToV2() 17
v3ShardingLayout() 18
shardChunkReferences() 18
v3ChunkKeyToV2() 18

zarrcurator Documentation

Welcome to zarrcurator documentation in a Read the Docs style layout.

Overview

zarrcurator analyzes Zarr stores for completeness and sizing, supports visualization, and can package selected content into tar archives for downstream archival workflows.

Build Documentation

From the repository root:

```
sphinx-build -b html docs docs/_build/html
sphinx-build -b pdf docs docs/_build/pdf
```

Generated artifacts:

- HTML index: docs/_build/html/index.html
- PDF: docs/_build/pdf/zarrcurator.pdf

Regenerate API Docstring Page

```
python scripts/generate_api_docstrings.py
```

Object Store Credentials

zarrcurator supports anonymous-first access for supported S3 endpoints. If a store is private, you can set credentials explicitly via utility APIs.

Preferred helper:

```
import zarrcurator

zarrcurator.setObjectStoreCredentials(
        accessKey="AWS_ACCESS_KEY_ID",  # or literal access key value
        secretKey="AWS_SECRET_ACCESS_KEY",  # or literal secret value
)
```

Per-call credentials are also accepted by store-facing utilities such as:

- diagnoseStore(…, accessKey=…, secretKey=…)
- findMissingChunksInStore(…, accessKey=…, secretKey=…)
- displayStoreSunburst(…, accessKey=…, secretKey=…)

The generated `.check` report from `findMissingChunksInStore` includes per-variable `missing`, `zero_size`, `corrupt`, and `unexpected` chunk lists.

Chunk keys are recognised for zarr v2 (`0.1`, `0/1`) and zarr v3 (`c/0/1`, `c.0.1`, and the v2-style encoding without the `c` prefix). In the report, `zero_size` and `corrupt` list the chunk keys as found in the store, while `missing` lists dot-separated chunk coordinates (e.g. `0.0.1`) regardless of layout. A zero-size chunk cannot be read, so it is also listed under `corrupt`. All arrays are checked, including those in nested groups; sharded arrays are checked by shard. See Store Layouts for zarr v2/v3, nested groups (DataTree) and sharding, and Known Issues and Limitations.

Synthetic Test Stores

`python -m zarrcurator.synthetic_store` creates small stores with injected anomalies for end-to-end checks:

- `--zarr-formats 2,3` selects the formats and `--separators .,/` the chunk key separators; without `--separators` each format uses its default (v2 `.`, v3 `/`).

- `--shards 4,4,8` also creates the zarr v3 stores with sharded data variables; the shard shape must be a multiple of the chunk shape `2,4,4`.
- `--layouts flat,tree` also creates stores with nested groups: `time` in the root group, `lat`, `lon`, `temp` and `humidity` in `/atmos`, and `salinity` in `/atmos/ocean`, which uses the coordinates of the groups above it.

The variant is tagged in the store name, only where it differs from the default, e.g. `syntheticexp_P1D_000_z2slash.zarr` or `syntheticexp_P1D_000_z3shardtree.zarr`. The `manifest.json` written next to the stores records each store's variant, the array the anomalies are injected into (`temp`, or `atmos/ocean/salinity` in a tree) and the anomalies, with paths relative to its own directory.

`dispatchStoreToTarStores(...)` supports both local-disk and object-store Zarr backends. It first checks that the store opens with xarray (`xarray.open_datatree`) and raises a `ValueError` otherwise, before any diagnosis or tarball is made.

Store Layouts

zarrcurator diagnoses, splits and archives zarr stores in all of these layouts, and creates Kerchunk references for the resulting tarballs:

Layout	Supported
zarr v2 and zarr v3	yes
Chunk key separators `.` and `/`	yes: v2 `0.1` / `0/1`, v3 `c.0.1` / `c/0/1`, and v3 with the `v2` key encoding
Nested groups (e.g. written with `xarray.DataTree.to_zarr`)	yes
Sharded arrays (zarr v3 `sharding_indexed`)	yes, see Sharding below

A store must open with xarray: every array needs dimension names (`_ARRAY_DIMENSIONS` in zarr v2, `dimension_names` in zarr v3). Stores written with xarray have them; stores written with plain zarr may not. `dispatchStoreToTarStores` checks this first (with `checkStoreOpensWithXarray`) and raises a `ValueError` before diagnosing, splitting or archiving a store xarray cannot open.

Nested groups

In a store with nested groups, each array is handled by its path from the store root, e.g. `atmos/ocean/salinity`.

- **Diagnosis** records the size of each array by its path, and the metadata files of each group, e.g. `atmos/.zgroup` (v2) or `atmos/zarr.json` (v3).
- `findMissingChunksInStore` checks every array in the store, including those in nested groups. The `.check` report is keyed by the array path, e.g. `/atmos/ocean/salinity`.
- **Splitting**: a group whose arrays together fit the target size goes into one tarball. A larger group is split into its arrays and its sub groups, which are handled the same way; an array larger than the target is split into tarballs of its chunks.
- **Coordinates**: each tarball gets the coordinate variables of its arrays. A group can use the coordinates of the groups above it, as in an xarray DataTree, so each coordinate is looked up in the array's own group first, then in its parent groups up to the store root.
- **Tarballs** contain the metadata of every group from the store root down to their arrays, so each tarball opens with `xarray.open_datatree`.

Sharding

In a sharded array each shard is one file, so zarrcurator handles shards like chunks:

- `findMissingChunksInStore` checks the shards (`expected_chunks` counts shards). A shard of zero size is reported under `zero_size`; zarr reads an empty shard as not written, so unlike a zero-size chunk it is not also reported as `corrupt`.

- Arrays larger than the target size are split into tarballs of whole shards. A shard cannot be split, so a single shard larger than the target size makes a tarball larger than the target.
- The Kerchunk references point to the inner chunks: the index of each shard is read from the tarball (at the end or the start of the shard, its `crc32c` checksum verified) and each inner chunk becomes a reference to its byte range. A corrupted shard index raises a `ValueError`.

Nested sharding, a codec applied after `sharding_indexed`, and shard index codecs other than `bytes` with an optional `crc32c` are not supported and raise a `ValueError`.

Kerchunk references

`createKerchunkReferenceFileForTar` writes the references of one tarball; `convertKerchunkReferenceFilesOfZarrStoreToParquet` merges the references of all tarballs of a store (`<store>_*_Offset_kerchunk.json`) into one parquet reference of the whole store (`<store>.parq`).

- The tarballs of a store are parts of it, so their references are merged: each chunk of a data variable is in one tarball; metadata and coordinate variables are in several and are kept once, after checking that their content is the same. A key with a different content in two tarballs raises a `ValueError`.
- The references are written in the zarr v2 layout. For zarr v3 stores the metadata is translated to zarr v2 and the chunks are referenced unchanged, which works for the `bytes` codec followed by at most one compressor (`zstd`, `gzip`, `blosc` or a `numcodecs.*` codec). The reopened data is zarr v2.
- The references of different stores are not combined: tarballs are not concatenated along `time`.

Opening the references of a store without nested groups:

```
import fsspec
import xarray as xr

mapper = fsspec.get_mapper(
    "reference://", fo="work/mystore.parq", remote_protocol="file"
)
ds = xr.open_dataset(mapper, engine="zarr", consolidated=False, zarr_format=2)
```

Opening the references of a store with nested groups needs `consolidated=True`, as fsspec's listing of parquet references does not find nested groups:

```
tree = xr.open_datatree(
    "reference://",
    engine="zarr",
    consolidated=True,
    zarr_format=2,
    storage_options={"fo": "work/mystore.parq", "remote_protocol": "file"},
)
```

Synthetic test stores

`python -m zarrcurator.synthetic_store` generates small stores in each of these layouts, with anomalies injected for the diagnostics, see the README. The fixture in `tests/data/synthetic_experiment` holds one store per layout and frequency.

Known Issues and Limitations

Known issues

- **Target sizes below 1 GiB fail for large arrays.** `Dispatcher.createTarForChunk` computes the number of tarballs in whole GiB (`divmod(int(size), int(targetSize))`), so a target size below 1 GiB raises `ZeroDivisionError` when an array exceeds the target size.

- **A metadata file larger than the target size fails.** The distributor treats it like a large array and passes it to `createTarForChunk`, which expects a directory. This can happen with the consolidated metadata (`.zmetadata`, or the root `zarr.json`) of a large store with many groups.
- `dispatch(freq, zoomLevel, subGroupID)` **fails for split stores.** It selects the subgroups of a store by the store name `<experiment>_<frequency>_<zoom>`, which it reads from the path of the first member of each subgroup. The members of `var` and `chunk` subgroups are named relative to the store (e.g. `tas`), so this raises `IndexError`; for an array in a nested group (e.g. `z2/ocean/sst`) it reads a group name instead and silently selects nothing. `dispatchStoreToTarStores` is not affected.
- **One zoom level per store.** `dispatch` relies on the naming convention of one HEALPix zoom level per store, so a store holding several zoom levels as groups cannot be archived per zoom level.
- **Unclosed reference files.** `convertKerchunkReferenceFilesOfZarrStoreToParquet` writes and reads the `*_Offset_kerchunk.json.copy` files without closing them. This works because CPython closes them immediately, and shows as `ResourceWarning` in the tests.

Limitations

- **Kerchunk references are zarr v2.** zarr v3 stores are referenced through a translation of their metadata to zarr v2; codecs without a zarr v2 equivalent (`transpose`, chained compressors) raise a `ValueError`. See Store Layouts.
- **Fill values of translated zarr v3 arrays.** xarray keeps the fill value of zarr v3 arrays in the `_FillValue` attribute. An array without it but with a non-zero zarr fill value keeps that fill value in the zarr v2 references, where xarray treats it as missing data; a warning is logged.
- **Nested groups need consolidated metadata.** Parquet references of a store with nested groups must be opened with `consolidated=True`: fsspec's listing of parquet references does not find nested groups.
- **Shards are not split.** A single shard larger than the target size makes a tarball larger than the target size.
- **Unsupported sharding.** Nested sharding, a codec after `sharding_indexed` and shard index codecs other than `bytes` with an optional `crc32c` raise a `ValueError`.

Worked around

- **Missing chunks in parquet references (fsspec).** When fsspec stores the URLs of an array's references as a pandas categorical (by default when an array has at least 10 references per URL, i.e. nearly every array of a tarball), a chunk missing from the references reads back as `[nan]` instead of as missing. zarr does not write chunks that hold only the fill value, so this gave wrong data. zarrcurator writes the parquet references without categoricals (`categorical_threshold=sys.maxsize`).

zarrcurator API Documentation

Generated from source docstrings.

Generated on: 2026-09-30

Module: init

Source: src/zarrcurator/**init**.py

No docstring provided.

Functions

enable_logging()

Enable zarrcurator logging for notebooks and library usage.

This configures both console logging and a timestamped log file, using the same behavior as the CLI.

Module: benchmark

Source: src/zarrcurator/benchmark.py

Micro-benchmark harness for zarrcurator.

Run examples: python -m zarrcurator.benchmark –synthetic python -m zarrcurator.benchmark –store-path /path/to/store.zarr –phase diagnose python -m zarrcurator.benchmark –synthetic –phase both –log-level INFO

Functions

create_synthetic_store()

Create a lightweight local zarr-like directory tree for benchmarking I/O paths.

benchmark_diagnose()

No docstring provided.

benchmark_dispatch()

No docstring provided.

parse_args()

No docstring provided.

main()

No docstring provided.

Classes

_BenchDistributor

No docstring provided.

Module: chunkkeys

Source: src/zarrcurator/chunkkeys.py

This module contains helpers for parsing zarr chunk keys.

Functions

parseChunkKey()

Parse the chunk coordinates from a chunk key relative to its variable.

Supports zarr v2 keys separated by ‘.’ (0.1) or ‘/’ (0/1), and zarr v3 keys with the ‘c’ prefix separated by ‘/’ (c/0/1) or ‘.’ (c.0.1).

Args: str: Chunk key relative to the variable directory. Returns: tuple: Chunk coordinates, or None if the key is not a chunk key.

Module: cli

Source: src/zarrcurator/cli.py

No docstring provided.

Functions

configure_logging()

Configure root logging for zarrcurator CLI usage.

Priority:

1. CLI –log-level value
2. ZARRCURATOR_LOG_LEVEL environment variable
3. INFO default

main()

No docstring provided.

Module: config

Source: src/zarrcurator/config.py

This module contains the configuration variables to handle the various features of the zarrcurator tool. The configuration variables consists of the various path, the flag to handle the mode of operation viz., interactive or batch mode.

Classes

configVars

Config class to hold the configuration variables to be used across the modules in the package.

Methods of configVars

- getZarrStorePath(): Get the path of the zarr stores. To get the directory under which the zarr stores, for each healpix and frequency combination exist. Args: None. Returns: path: Path to zarr stores.
- setZarrStorePath(): Set the path of the zarr stores. To set the directory under which the zarr stores, for each healpix and frequency combination exist. Args: path: Path to zarr stores. Returns: None.
- getFileSystemObj(): Get the file system object. To get the file system object, so that it can be used in other classes of the zarrcurator viz., diagnoser. Args: None. Returns: object[class FileSystem]: File system object.
- setFileSystemObj(): Set the file system object. To set the file system object, so that it can be used in other classes of the zarrcurator viz., diagnoser. Args: None. Returns: None.
- parse_object_store_url(): Parse an object-store HTTP URL into: - endpoint - bucket - object key - s3 uri
- getWorkingDir(): Get the path of the working directory. To get the directory under which the output produced by zarrcurator will be stored. Args: None. Returns: path: Path to working directory.
- setWorkingDir(): Set the path of the working directory. To set the directory under which the output produced by zarrcurator will be stored. Args: path: Path to working directory. Returns: None.
- getArchBasePath(): Get the path of the directory on HSM. To get the location of the directory on HSM under which the tarballs prouced by zarrcurator tool will be stored. Args: None. Returns: path: Path to base directory on HSM.
- setArchBasePath(): Set the path of the directory on HSM. To set the location of the directory on HSM under which the tarballs prouced by zarrcurator tool will be stored. Args: path: Path to base directory on HSM. Returns: None.
- getJsonFile(): Get the name of the JSON file. To get the name of the output JSON file produced by zarrcurator. Args: None. Returns: str: Name of the JSON file.
- setJsonFile(): Set the name of the JSON file. To set the name of the output JSON file produced by zarrcurator. Args: str: Name of the JSON file. Returns: None.

- getTargetSize(): Get the target size of the tarball. To get the target size of the tar ball keeping in view the constraints of the HSM system, as of now 200 GB. Args: None. Returns: int: Target size of the tarball.
- setTargetSize(): Set the target size of the tarball. To set the target size of the tar ball keeping in view the constraints of the HSM system, as of now 200 GB. Args: int: Target size of the tarball. Returns: None.
- getBatchMode(): Get the mode of operation of the zarrcurator. To get the mode of operation of the zarrcurator viz., interactive or batch mode. Args: None. Returns: bool: mode of operation of the zarrcurator.
- setBatchMode(): Set the mode of operation of the zarrcurator. To set the mode of operation of the zarrcurator viz., interactive or batch mode. Args: bool: mode of operation of the zarrcurator. Returns: None.
- getDebugMode(): Get the debug mode of operation of the zarrcurator. To get the debug mode of operation of the zarrcurator. Args: None. Returns: bool: Debug mode of operation of the zarrcurator.
- setDebugMode(): Set the debug mode of operation of the zarrcurator. To set the debug mode of operation of the zarrcurator. Args: bool: Debug mode of operation of the zarrcurator. Returns: None.
- getPackDiscontinuousData(): Get the flag for packing the discontinuous data. To get flag to pack the data even if the chunks are missing. Args: None. Returns: bool: Flag to indicate the packing of missing chunks.
- setPackDiscontinuousData(): Set the flag for packing the discontinuous data. To set the flag to pack the data even if the chunks are missing . Args: bool: Flag to indicate the packing of missing chunks. Returns: None.
- getaccessKey(): Get the accessKey for accessing the object store. To get the access key environment variable name. Args: None. Returns: str: Name of the environment variable.
- setaccessKey(): Set the accessKey for accessing the object store. To set the access key environment variable name. Args: str: Name of the environment variable. Returns: None.
- getsecretKey(): Get the secretKey for accessing the object store. To get the secret key environment variable name. Args: None. Returns: str: Name of the environment variable.
- setsecretKey(): Set the secretKey for accessing the object store. To set the secret key environment variable name. Args: str: Name of the environment variable. Returns: None.
- getDistributorObj(): Get the distributor object. To get the distributor object, so that it can be used in other classes of the zarrcurator viz., dispatcher. Args: None. Returns: object[class Distributor]: Distributor object.
- setDistributorObj(): Set the distributor object. To set the distributor object, so that it can be used in other classes of the zarrcurator viz., dispatcher. Args: object[class Distributor]: Distributor object. Returns: None.
- getDiagnoserObj(): Get the diagnoser object. To get the diagnoser object, so that it can be used in other classes of the zarrcurator viz., displayer. Args: None. Returns: object[class Diagnoser]: Diagnoser object.
- setDiagnoserObj(): Set the diagnoser object. To set the diagnoser object, so that it can be used in other classes of the zarrcurator viz., displayer. Args: object[class Diagnoser]: Diagnoser object. Returns: None.
- getDispatcherObj(): Get the dispatcher object. To get the dispatcher object, so that it can be used in other parts of the zarrcurator viz., utils. Args: None. Returns: object[class Dispatcher]: Dispatcher object.
- setDispatcherObj(): Set the dispatcher object. To set the dispatcher object, so that it can be used in other parts of the zarrcurator viz., utils. Args: object[class Dispatcher]: Dispatcher object. Returns: None.
- toFile(): Save the config object to a JSON file. To save the zarrcurator configuration object, so that it can be edited and used. Args: str: File name for storing the config object in JSON format. Returns: None.
- fromFile(): Read the config object from a JSON file. To read the zarrcurator configuration object, so that it can be used in the current session. Args: str: File name for reading the config object in JSON format. Returns: None.

Module: core

Source: src/zarrcurator/core.py

No docstring provided.

Functions

hello()

No docstring provided.

helloFrom()

No docstring provided.

Module: diagnoser

Source: src/zarrcurator/diagnoser.py

This module contains the declaration and definition of the class Diagnoser.

Classes

Diagnoser

Class to Diagonose the given experiment / zarr store / variable etc and gather the file sizes.

Methods of Diagnoser

- get_list_of_vars(): Get the variables in a given zarr store. For getting the list of variables in a given zarr store, by recursively scanning the zarr store directory. Args: path:Path to the given zarr store. Returns: list: List of variables found in the given store.
- get_dir_size(): Get the appdataexplorer installation path. For getting the location where the appdataexplorer is installed on the disk. Args: path: Path to appdataexplorer. Returns: int,int: The cumulative size of the files and their count.
- findChunkSize(): Check the size of a single zarr chunk file. Find the size of zarr chunk file and update the missing chunks list. Args: path: Path to zarr chunk. Returns: None.
- findMissingChunksInZarr(): Diagnose a single zarr store to find zero size files i.e., missing chunks. For diagnosing a single zarr store and identify the missing chunks. Args: path: Path to zarr store. Returns: list:List of missing chunks.
- find_size(): Get the size of the file in the variable path. For finding the size of the content in the variable under the path provided. Args: path: Path to appdataexplorer. Returns: None.
- diagnose_var(): Diagnose a single variable in a zarr store. For diagnosing a single variable in a zarr store. Args: str: Name of the variable. path: Path to zarr store. Returns: int,int,int:Cumulative size, count and time taken for diagnosis.
- expand_groups(): Replace the sub groups among the entries of a store by their own entries. For stores with nested groups, e.g. written from an xarray DataTree, so that each array is diagnosed on its own and named by its path from the store root. Args: list: Entry paths (local store) or fsspec detail dicts (S3 store). str: Path prefix of the entries relative to the store root, e.g. ‘z2/’. Returns: list[tuple]: (entry, name) pairs, name being the path from the store root.
- diagnose_zarr_var(): Diagnose a single entry (variable or metadata file) of a zarr store. Args: var: Entry path (local store) or fsspec detail dict (S3 store). zarr_path: Path to zarr store. var_name: Name to report the entry by, e.g. its path ‘z2/ocean/sst’ in a store with nested groups; defaults to the entry’s own name. Returns: tuple: (var_name, var_size, var_chunk_count, var_time).
- diagnose_zarr(): Diagnose a single zarr store. For diagnosing a single zarr store. When ZARRCURATOR_MULTITHREAD=1, the variables of the store are diagnosed in parallel. Arrays in nested groups are recorded by their path from the store root, e.g. ‘z2/ocean/sst’, and each group’s metadata files likewise, e.g. ‘z2/.zgroup’. Args: path: Path to zarr store. Returns: int:Time in secs for diagnosing.
- diagnose_experiment(): Diagnose the experiment. For diagnosing all the zarr stores in parallel using multithreads. Args: None. Returns: None.

- update(): To update the diagnoser object with the data obtained from the diagnosis of the input zarr stores. The diagnoser object will be updated either by scanning each of the zarr stores in the input experiment path, if starting afresh or use the JSON file created from a previous diagnosis of the experiment. Args: None. Returns: None.
- createStatistics(): Create statistics of the output from the entire experiment. Generate some statistics like min, max file sizes, counts etc. Args: None. Returns: None.

Module: dispatcher

Source: src/zarrcurator/dispatcher.py

This module contains the declaration and definition of the class Dispatcher.

Classes

Dispatcher

Class to Dispatch the diagnoser output to create tar balls.

Methods of Dispatcher

- createTarForFrequencySubGroupDict(): Create the tar balls for all the zarr stores. For creating tar balls for all the zarr stores in the experiment and listed in the 'frequencySubgroupDict' dictionary created by the diagnoser. Args: None. Returns: None.
- createTarForFrequencySubGroups(): Create the tar balls for all the zarr stores in a given frequency. For creating tar balls for all the zarr stores in the experiment for a given frequency listed in the 'frequencySubgroupDict' dictionary created by the diagnoser. Args: str: Frequency for which the tar balls are to be created. Returns: None.
- createTarForFrequencyHealPixLevelSubGroup(): Create the tar balls for all the zarr stores in a given frequency and heal pix level. For creating tar balls for all the zarr stores in the experiment for a given frequency and healpix level, listed in the 'frequencySubgroupDict' dictionary created by the diagnoser. Args: str: Frequency for which the tar balls are to be created. int:Healpix level for which the tar balls are to be created. Returns: None.
- createTarForSubGroup(): Create the tar balls for a sub group. For creating tar balls for a sub group based on the type and name them with a specific name. Args: object[Class Subgroup]: Sub group object. str:Name of the tar ball that will be created for the subGroup. Returns: None.
- createTarForZarr(): Create the tar balls for a sub group of type 'zarr'. For creating tar balls for 'zarr' type sub group based on the type and and name them with a specific name. Args: object[Class Subgroup]: Sub group object. str:Name of the tar ball that will be created for the subGroup. Returns: None.
- createTarForVar(): Create the tar balls for a sub group of type 'var'. For creating tar balls for 'var' type sub group based on the type and and name them with a specific name. Args: object[Class Subgroup]: Sub group object. str:Name of the tar ball that will be created for the subGroup. Returns: None.
- createTarForChunk(): Create the tar balls for a sub group of type 'chunk'. For creating tar balls for 'chunk' type sub group based on the type and and name them with a specific name. Args: object[Class Subgroup]: Sub group object. str:Name of the tar ball that will be created for the subGroup. Returns: None.
- executor(): Create the tar balls. For creating tar balls for all the subgroups with their respective names. Args: int:Index of the subgroup for which the tar balls are to be created. Returns: None.

Module: displayer

Source: src/zarrcurator/displayer.py

This module contains the declaration and definition of the class Displayer.

Classes

Displayer

Class to create plots for the groups created using the distributorObj and diagnoserObj.

Methods of Displayer

- make_autopct(): No docstring provided.
- createPiePlotsForSubGroups(): Create Pie chart plots for the subgroups. For creating pie chart plots for the subgroups based on the sub group distribution method. Args: None. Returns: None.
- createHBarPlotsForExp(): Create horizontal bar plots for the subgroups. For creating horizontal bar plots for the entire zarr stores in an experiment. The plots can be based on file count or file sizes. Args: bool:Flag to indicate if the count of the files is represented on the axis. bool:Flag to indicate if the sizes of the files is represented on the axis. Returns: None.
- createHBarPlotForStoreVariables(): Create horizontal bar plots for the sizes of the variables for a given zarr store. For creating horizontal bar plots for the entire variables in a zarr store. The plots can be based on file count or file sizes. Args: bool:Flag to indicate if the count of the files is represented on the axis. bool:Flag to indicate if the sizes of the files is represented on the axis. Returns: None.
- createHBarPlotForStoreTarTapes(): Create horizontal bar plots for the sizes of the variables for a given zarr store. For creating horizontal bar plots for the entire variables in a zarr store. The plots can be based on file count or file sizes. Args: bool:Flag to indicate if the count of the files is represented on the axis. bool:Flag to indicate if the sizes of the files is represented on the axis. Returns: None.

Module: distributor

Source: src/zarrcurator/distributor.py

This module contains the declaration and definition of the class Distributor.

Classes

Distributor

Distribute the list of tuples of type (variable ,size) into different sub groups, either equal size or based on target size specified by user etc.

Methods of Distributor

- setFrequencySubGroupDict(): To set the frequency subGroup dictionary directly from JSON file. To set frequencySubGroupDict directly, if for example, loaded from JSON file from an earlier run of ‘zarrcurator’. Args: frequencySubGroupDict[dict]: Dictionary of the frequencies and subGroups. Returns: None.
- getFrequencySubGroupDict(): To get the frequency subGroup dictionary directly. To get frequencySubGroupDict directly, if for example, from utils of ‘zarrcurator’. Args: None. Returns: frequencySubGroupDict[dict]: Dictionary of the frequencies and subGroups.
- updateFrequencyGroupDict(): To create the dictionary per frequency and it’s groups. For storing the file sizes and counts for each zarr store per frequency in a dictionary. Args: None. Returns: None.
- updateFrequencySubGroupDictWithSingleZarrStore(): To update the sub group dictionary per frequency with single subgroup of type ‘zarr’. For initializing each of the zarr stores (per freq and healpix) in the sub group dictionary with sub group of type ‘zarr’, if the size of the entire zarr store is less than the target size. Args: None. Returns: None.
- updateFrequencySubGroupDictWithTargetSize(): To update the sub group dictionary per frequency with subgroups of size as per target size. For updating each of the zarr stores (per freq and healpix) in the sub group dictionary with sub group whose size is greater than the target size and splitting the sub groups into more subgroups if larger than the target size. Args: None. Returns: None.

- groupStoreEntries(): To group the entries of a zarr store with nested groups into units that are kept together. For stores with nested groups (e.g. written from an xarray DataTree), the entries are named by their path from the store root, e.g. ‘z2/ocean/sst’. A group whose entries in total fit the target size is kept together as one unit, a larger group is split into its own entries and its sub groups, which are handled the same way. Entries at the store root are units of their own, so for a store without nested groups every entry is a unit. Args: dict: Entry name -> {‘size’, ‘count’} for the entries of a zarr store. Returns: list[tuple]: (list of entry names, total size) per unit.
- split_zarr_into_equalSized_bins(): To update the sub group dictionary per frequency with subgroups of size as per target size. For updating each of the zarr stores (per freq and healpix) in the sub group dictionary with sub group whose size meets the target size and splitting the sub groups into more subgroups if larger than the target size. Args: obj:. Returns: None.
- redistributeSubGroups(): To redistribute the subgroups. For redistributing each of the zarr stores (per freq and healpix) in the sub group dictionary based on each of the sub group and it’s size in comparison with the target size,by splitting the sub groups into more subgroups if their size is larger than the target size. Args: obj: None. Returns: None.
- updateDimensionsToSubGroups(): To update the dimensions to the subgroups. For each of the sub group dictionary based on it’s type, for the ‘var’ and ‘chunk’ types, the dimensions of the original zarr store are to be appended, so that the tar ball is self-contained and could be utiized by the downstream applications. Args: obj: None. Returns: None.
- getZarrstoreDimensions(): To get the coordinate variables of the zarr store. For getting the coordinate variables of the original zarr store, which are to be appended to the tar balls. These are the dimension coordinates (arrays named after a dimension, e.g. ‘time’ or the HEALPix ‘cell’), the auxiliary coordinates listed in the ‘coordinates’ attribute and the variables named in the ‘grid_mapping’ attribute. Dimension names are read from ‘_ARRAY_DIMENSIONS’ (zarr v2) or ‘dimension_names’ (zarr v3). For stores with nested groups (e.g. written from an xarray DataTree), the coordinates used by the arrays of each of the given groups are collected. As a group can use the coordinates of its parent groups, each coordinate is looked up in the group itself first and then in its parents up to the store root. Args: str: Frequency of the model data. str: Path to the location of the zarr store. list[str]: Group paths relative to the store root, e.g. [‘z2/ocean’]; defaults to the root group ‘’. Returns: list: sorted list of (coordinate variable path, size) tuples.
- update(): To update the distributor object with the data obtained from the distribution of the input zarr stores. The distributor object will be updated either by scanning each of the zarr stores in the input experiment path, if starting afresh or use the JSON file created from a previous distribution of the experiment. Args: None. Returns: None.

Module: oceanVarsYAML

Source: src/zarrcurator/oceanVarsYAML.py

No docstring provided.

Module: storetree

Source: src/zarrcurator/storetree.py

This module contains helpers for walking the group hierarchy of a zarr store, e.g. a store written from an xarray DataTree, with nested groups.

Functions

isZarrGroupDir()

Check whether a directory is a zarr group.

A zarr v2 group holds a ‘.zgroup’ file, a zarr v3 group a ‘zarr.json’ file with node_type ‘group’. Array directories and plain directories are not groups.

Args: path[str]: Path of the directory. fs[object]: fsspec file system object for object stores, None for local stores. Returns: bool: True if the directory is a zarr group.

listArrayPaths()

List the paths of all arrays in a zarr group and its sub groups.

Works for zarr v2 and v3 groups, with or without consolidated metadata.

Args: group[zarr.Group]: Opened zarr group, usually the root of the store. Returns: list[str]: Sorted array paths relative to the group, e.g. [‘time’, ‘z2/ocean/sst’].

groupAncestors()

List the groups enclosing a store member, from the root down.

Args: path[str]: Path of the member relative to the store root, e.g. ‘z2/ocean/sst’. Returns: list[str]: Group paths relative to the store root, ‘’ being the root, e.g. [‘’, ‘z2’, ‘z2/ocean’].

Module: subgroup

Source: src/zarrcurator/subgroup.py

This module contains the declaration and definition of the class SubGroup.

Functions

subGroup_to_dict()

No docstring provided.

subGroup_from_dict()

No docstring provided.

Classes

SubGroup

Class to create subGroups of type ‘zarr’, ‘var’ or ‘chunk’ based on the size of the zarr store.

Methods of SubGroup

- groupSize(): To get the group size of the subGroup. To get the type of the subGroup. Args: None. Returns: int: Size of the subGroup.
- groupSize(): To compute the size of the subGroup. To compute the size of the subGroup by adding up the subGroup members. Args: int: To set the groupsize to the variable. Returns: None.
- setGroupType(): To set the type of the subGroup. To set the type of the subGroup. Args: str: Type of the subGroup ‘zarr’, ‘var’ or ‘chunk’. Returns: None.
- getGroupType(): To get the type of the subGroup. To get the type of the subGroup. Args: None. Returns: str: Type of the subGroup ‘zarr’, ‘var’ or ‘chunk’.
- setGroupTarMembers(): To set the tar file names for the subGroup. To set the list of tar file names for the tarballs produced for the subGroup. Args: list[str]: List of tar file names. Returns: None.
- getTarMembers(): To get the tar file names for the subGroup. To get the list of tar file names for the tarballs produced for the subGroup. Args: None. Returns: list[str]: List of tar file names.
- setgroupTarTapeIDs(): To set the tape IDs for the tar file names for the subGroup. To set the list of tape IDs for the tar file names for the tarballs produced for the subGroup. Args: list[str]: List of tape IDS for tar file names. Returns: None.
- getTarTapeIDs(): To get the tape IDs for the tar file names for the subGroup. To get the list of tape IDs for the tar file names for the tarballs produced for the subGroup. Args: None. Returns: list[str]: List of tape IDs for tar file names.

- updateSubGroup(): To update the subGroup object with diagnoser object with the data obtained from the diagnosis of the input zarr stores. The diagnoser object will be updated either by scanning each of the zarr stores in the input experiment path, if starting afresh or use the JSON file created from a previous diagnosis of the experiment. Args: list[tuple] or tuple: Tuple or list of tuples to update the SubGroup object. Returns: None.
- updateSubGroupSize(): To update the subGroup size. To update the subgroup size by adding up the sizes of the subGroup members. Args: None. Returns: None.

Module: synthetic_store

Source: src/zarrcurator/synthetic_store.py

Synthetic Zarr experiment generator for zarrcurator.

This module creates local Zarr stores with predictable naming and metadata so core zarrcurator workflows can be tested end-to-end.

Examples: python -m zarrcurator.synthetic_store –output-dir tests/data/synthetic_experiment python -m zarrcurator.synthetic_store –output-dir tests/data/synthetic_experiment –run-smoke python -m zarrcurator.synthetic_store –output-dir /tmp/layouts –separators .,/ python -m zarrcurator.synthetic_store –output-dir /tmp/all –separators .,/ –shards 4,4,8 –layouts flat,tree

Functions

store_variant_tag()

Return the tag of a store variant in store names, e.g. ‘z2’, ‘z2slash’, ‘z3shard’, ‘z3dotshardtree’.

Only what differs from the default (format default separator, no sharding, flat) is added. The tag has no *‘’, as store names are split on ‘’* by the dispatcher.

create_synthetic_experiment()

Create an experiment directory containing multiple synthetic stores.

One set of stores is created per combination of zarr format, entry in `separators` (None: only the format default), sharding and entry in `layouts` (None: only ‘flat’). With `shards`, the zarr v3 stores are created both without and with sharding of the data variables. The variant is tagged in the store name, see store_variant_tag(), e.g. `syntheticexp_P1D_000_z2slash.zarr` or `syntheticexp_P1D_000_z3shardtree.zarr`.

run_smoke_pipeline()

Run diagnose/distribute/dispatch on the generated synthetic experiment.

parse_args()

No docstring provided.

main()

No docstring provided.

Module: utils

Source: src/zarrcurator/utils.py

This module contains certain utility functions that will be exposed to the user to perform various tasks.

Functions

usage()

Help about the usage of zarrcurator. Returns: None.

saveConfigToFile()

Saving the zarrcurator config to file.

Saving the zarrcurator library configuration as a JSON file.

Args: configFileName : File name in which to store the configuration. Returns: None.

loadConfigFromFile()

Loading the zarrcurator config from file.

Reading the zarrcurator library configuration from a JSON file.

Args: configFileName : File name in which to store the configuration. Returns: None.

setObjectStoreCredentials()

Set object-store credentials for the current process.

Accepts either raw key values or environment variable names that contain those values. Resolved values are exported to AWS/S3 standard environment variables and wired into configObj for downstream filesystem creation.

Args: accessKey[str]: Access key value or env-var name containing it. secretKey[str]: Secret key value or env-var name containing it. Returns: None.

updateStoretoConfig()

Update the store location to the config object.

To update the store location to the config object, to be used by other functions in the zarrcurator library.

Args: storeLocation[str]: Location of the zarr store. accessKey[str]: Optional access key value/env-var name. secretKey[str]: Optional secret key value/env-var name. Returns: None.

findMissingChunksInStore()

Finding missing chunks in a zarr store.

To find chunks in store that are missing, artifacts, corrupt or zero sized. All arrays are checked, including those in nested groups (e.g. a store written from an xarray DataTree). When ZARRCURATOR_MULTITHREAD=1, chunk size queries and the chunk readability (corruption) check run in parallel threads.

Args: storePath : Path of the zarr store. accessKey[str]: Optional access key value/env-var name. secretKey[str]: Optional secret key value/env-var name. Returns: None.

initialize()

Initializing the zarrcurator.

Initialize the zarrcurator library configuration with the paths, flags etc.

Args: expPath : Directory under which the simulation data (zarr stores) exists. workDir : Working directory where the output will be stored. jsonFile : Name of the JSON file in which the output of the zarrcurator will be stored. targetSize : Target size of the tarball keeping in view of the HSM constraint (200 GB as of now). batchMode : Flag to indicate if mode of operation of the zarrcurator is batch mode. debugMode : Flag to indicate if mode of operation of the zarrcurator is set to debugging. packDiscontinuousData : Flag to indicate if data with missing chunks are to be packed. accessKey[str]: Optional access key value/env-var name. secretKey[str]: Optional secret key value/env-var name.

Returns: None.

diagnose()

Perform the diagnosis of the input directory containing the various zarr stores.

Creates the object of class Diagnoser, to explore the contents of each zarr store and gather the count, sizes of the files inside them.

Args: None.
 \ Returns: None.

diagnoseStore()

Perform the diagnosis of the input zarr store.

To find the sizes, number of chunks for each of the variables inside the store.

Args: storeLocation[str]: Location of the zarr store. accessKey[str]: Optional access key value/env-var name. secretKey[str]: Optional secret key value/env-var name. Returns: None.

display()

Displays the output of the diagnoser object, by creating various plots.

Creates the object of class Displayer, to create bar plots, pie charts of the output generated by the diagnoser object viz., count, sizes of the files for each of the input zarr store, the constraints of the HSM system.

Args: None.
 \ Returns: None.

running_in_jupyter()

No docstring provided.

displayStoreSunburst()

Displays the output of the diagnoser object, by creating sunburst plot for the input zarr store.

Creates the object of class Displayer, to create sunburst plot of the output generated by the diagnoser object viz., count, sizes of the files for each of the variables in the input zarr store.

Args: storeLocation[str]: Location of the zarr store. accessKey[str]: Optional access key value/env-var name. secretKey[str]: Optional secret key value/env-var name. Returns: None.

displayZarrStoreVariables()

Displays the output of a given zarr store from the diagnoser object , by creating variable sizes plot.

Creates create bar plots of the input zarr store from output generated by the diagnoser object viz., count, sizes of the files for each of the variable in the input zarr store.

Args: zarrStore[str]: Name of the zarr store. fileCount[int]: Flag to indicate if file counts are to be plotted. byteBins[int]: Flag to indicate if file sizes are to be plotted. Returns: None.

distribute()

Distribute the output of the diagnoser object into subgroups of ‘zarr’, ‘var’ and ‘chunk’.

Creates the object of class Distributor, to create subgroups of ‘zarr’, ‘var’ and ‘chunk’, based on the size of the contents of the zarr store and the constraints of the HSM system.

Args: None.
 \ Returns: None.

distributeStoreToSubStores()

Distribute the output of the diagnoser object into subgroups.

Creates the object of class Distributor, to create subgroups of ‘zarr’, ‘var’ and ‘chunk’, based on the size of the contents of the zarr store and the constraints of the HSM system.

checkStoreOpensWithXarray()

Check that a zarr store can be opened with xarray, with all its groups.

The tarballs are read back with xarray, directly or through the Kerchunk references, which needs the dimension names of the arrays (‘_ARRAY_DIMENSIONS’ in zarr v2, ‘dimension_names’ in zarr v3). Stores written with plain zarr may lack them.

Args: storeTarget: Path of a local zarr store, or zarr store object of an object store. storeLocation[str]: Location of the zarr store, for the error message. Returns: None. Raises: ValueError: If xarray cannot open the store.

dispatchStoreToTarStores()

Dispatch the output of the distributor object into tarballs.

Creates the object of class Dispatcher, to create tarballs for each of the subgroups of ‘zarr’, ‘var’ and ‘chunk’. Fails with a ValueError before diagnosing, distributing or creating tarballs if the store cannot be opened with xarray, see checkStoreOpensWithXarray().

dispatch()

Dispatch the output of the distributor object into tarballs.

Creates the object of class Dispatcher, to create tarballs for each of the subgroups of ‘zarr’, ‘var’ and ‘chunk’.

Args: freq[str]: Frequency of the model output.
 \ zoomlevel[int]: Heal pix level. int:Index of the subgroup for which the tar balls are to be created. Returns: None.

createKerchunkReferenceFileForTar()

Create kerchunk reference file for given tar file.

To generate the kerchunk reference file in JSON format for the contents of the input tar archive.

Args: tarFileName[str]: Path of the tar archive. Returns: None.

mergeKerchunkReferences()

Merge the Kerchunk references of the tarballs of a zarr store into the references of the store.

The tarballs of a zarr store are parts of it: each holds some of its variables, or some of the chunks of a variable, along with the metadata of the store, its groups and its arrays and the coordinate variables it needs. So the references of the store are the union of the references of its tarballs. A chunk of a data variable is in one tarball; metadata and the chunks of coordinate variables are in several, and are kept once after checking that their content is the same. The consolidated metadata (‘.zmetadata’) is left out, as the parquet references hold their own.

Args: jsonFiles[list[str]]: Paths of the Kerchunk reference files (JSON) of the tarballs. Returns: dict: Merged references, key -> metadata content or [url, offset, size]. Raises: ValueError: If a key is found in two tarballs with a different content or location.

convertKerchunkReferenceFilesOfZarrStoreToParquet()

Convert given Kerchunk reference file to parquet file.

To generate the Kerchunk reference file in JSON format for the contents of the input tar archive. The references of all tarballs of the store are merged, see mergeKerchunkReferences(). The references are combined in the zarr v2 layout: for zarr v3 stores the zarr.json metadata is translated to zarr v2 metadata and the chunks are referenced unchanged, so only zarr v3 arrays with a zarr v2 equivalent are supported; sharded arrays are referenced by their inner chunks, read from the shard indexes. Stores with nested groups (e.g. written from an xarray DataTree) are opened from the parquet references with consolidated=True.

Args: tarFile[str]: Path of the tar archive. Returns: None.

updateArchBasePath()

Update the base path of the tape archive.

To set the base path on the tape archive under which the tarballs created for each of the subgroups of ‘zarr’, ‘var’ and ‘chunk’ are stored.

Args: archBasePath[str]: Path on the tape archive.
 \ Returns: None.

updateTarToTapeMapper()

Update the mapper for tarfiles and tape IDs.

To update the mapper which maps the input tar file to the ID of the tape on which this tar file resides.

Args: tarFile[str]: Path on the tar file.
 \ Returns: None.

mapTapeIDsToFrequencySubGroupDict()

Update the tape IDs to the FrequencySubGroupDict.

Creates the list of tape IDs for the tarballs of each subgroup in the FrequencySubGroupDict. To be called after the tarballs are transferred to the HSM tapes.

Args: None Returns: None.

displayZarrStoresTarTapes()

Displays the tar ball count and tape count for the zarr stores.

Creates create bar plots for the tar balls and tape counts for all the zarr stores.

Args: None. Returns: None.

displayZarrStoreVarTarTapes()

Displays the tar ball count and tape count for a given variable of a zarr store.

Creates create bar plots for the tar balls and tape counts for a particular variable from a zarr store.

Args: zarrStoreInput[str]: Name on the zarr store. varInput[list(str)]: List of names of the variables. Returns: None.

Module: zarrmetadata

Source: src/zarrcurator/zarrmetadata.py

This module contains helpers for translating zarr v3 metadata (zarr.json) to zarr v2 metadata (.zgroup, .zarray, .zattrs), so that the chunks of zarr v3 stores can be referenced by Kerchunk, which combines references in the zarr v2 layout.

The chunk bytes are referenced unchanged, so only codecs with a zarr v2 equivalent are supported: the ‘bytes’ codec followed by at most one compressor. ‘transpose’ and chained compressors raise a ValueError.

Sharded arrays (‘sharding_indexed’ codec) are translated to zarr v2 arrays of their inner chunks: each shard ends (or starts) with an index of the offset and size of its inner chunks, which is read to reference each inner chunk as a byte range within the shard. The inner codecs follow the rules above; nested sharding raises a ValueError.

Functions

v3CodecsToV2()

Translate the codecs of a zarr v3 array to a zarr v2 compressor.

Args: list[dict]: The ‘codecs’ of the zarr v3 array metadata. Returns: tuple: (endian, compressor), endian being ‘little’ or ‘big’ and compressor the zarr v2 compressor configuration or None.

v3MetadataToV2()

Translate the zarr v3 metadata of a group or array to zarr v2 metadata files.

The dimension names become the ‘_ARRAY_DIMENSIONS’ attribute. xarray keeps the fill value in the ‘_FillValue’ attribute of zarr v3 arrays, in zarr v2 it is the array’s fill_value, which xarray reads as ‘_FillValue’. So an array without ‘_FillValue’ and a zero fill value gets no fill value; zarr v2 reads missing chunks (not written as they hold only the fill value) as zeros then as well. A non-zero fill value is kept so missing chunks read correctly, though xarray then masks it.

Args: dict: Content of a zarr v3 ‘zarr.json’ file. Returns: dict: zarr v2 file name (‘.zgroup’, ‘.zarray’, ‘.zattrs’) -> content.

v3ShardingLayout()

Get the layout of the shards of a sharded zarr v3 array.

Args: dict: Content of the zarr v3 ‘zarr.json’ file of an array. Returns: dict or None: None for an array without sharding, else ‘shape’ (of the array), ‘shard_shape’, ‘chunk_shape’ (of the inner chunks), ‘codecs’ (of the inner chunks), ‘index_location’ (‘start’ or ‘end’), ‘index_endian’, ‘index_checksum’ (bool) and ‘index_size’ (bytes).

shardChunkReferences()

Get the references of the inner chunks of a shard.

Args: list: Reference of the shard, [url, offset, size]. str: Dot-separated zarr v2 key of the shard, relative to the array, e.g. ‘1.0’. dict: Layout of the shards, see v3ShardingLayout(). callable: read_range(url, start, end) returning the bytes in [start, end) of url. Returns: list[tuple]: (dot-separated key of the inner chunk relative to the array, [url, offset, size]) for every inner chunk in the shard that is written and lies within the array. Raises: ValueError: If the checksum of the shard index does not match.

v3ChunkKeyToV2()

Translate a chunk key of a zarr v3 array, relative to the array, to the dot-separated zarr v2 form.

Args: str: Chunk key, e.g. ‘c/0/1’ or ‘c.0.1’ (‘default’ encoding) or ‘0/1’ (‘v2’ encoding). dict: The ‘chunk_key_encoding’ of the zarr v3 array metadata. Returns: str: zarr v2 chunk key, e.g. ‘0.1’.